Assessment Tool - 1

Question 1 : online sympasium portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form is missing important semantic elements that improve webpage structure and accessibility.

<header> - Defines the heading section of the webpages
<main> - contains the main content.
<section> - Groups related content
<footer> - Defines the footer section
<label> - provides labels for form inputs.

2. modify the structure to make it standards-compliant.

```
<body>
  <header>
   <h1> online symposium Registration</h1>
  </header>
  <main>
    <section>
      <h2> Student Registration </h2>
      <form action = "/register" method = "post">
          <label for = "name"> Name : </label>
          <input type = "text" id = "name" name =
                "name" required >
      <label for = "email"> Email : </label>
      <input type = "email" id = "email" name = "email"
                                          required >
```

<button type = "subit > Submit </button>

</form>
</section>
</main>

3. Interpret the HTTP request generated during for submission

When the student clicks the submit button, the browser sends an HTTP POST request to the server

example:

```
http
post /register HTTP/1.1
HOST : example.com
content-Type : application/x-www-form-urlencoded
name = Spandana &email=Spandana%40gmail.com
```

Explanation:

POST - sends form data to the server

/register - The server endpoint the receives the data

HOST - Identifies the destination server

content-type - Specifies the format of the submitted data

Request body - contains the student's name and email.

4. Analyze the HTTP response cycle from server to browser.

1. The student fills in the registration form
2. The browser validates the form fields.

3. The browser Sends the post request to the server

4. The server receives and processes the submitted data

5. the server sends an HTTP response with a status code and response content.

6. The browser displays the response to the student.

Example : HTTP/1.1 200 OK means the request was processed successfully. The server may return a confirmation page.

5. Suggest improvements for usability and accessibility

1. Add <label> elements for every input field.
2. use the required attribute to prevent empty submissions.
3. use type="email" to validate email format
4. use clear headings error and success messages
5. Add helpful error and success messages
6. use appropriate colors, readable fonts, and keyboard - accessible controls.

2) Question - cross Browser compatibility Issue

1. Identify improper nesting and missing closing tags.

code :

```
<html>
<head>
```

```
<title> company </title>
</head>
<body>
 <div>
    <h1> welcome
    <p> about us
 </div>
</body>
</html>
```

Problems:

The <h1> element is missing its closing </h1> tag

The element is missing its closing tag

The document is missing the <!DOCTYPE html> declaration

The <html> element is missing the lang attribute

The code does not use semantic elements such as <header> and <main>

2) Analyze how different browsers may interpret this structure

1. Browsers may automatically insert missing closing tags.
2. the page may appear differently depending on the browser's HTML error-recovery behaviour.
3. the heading and paragraph may not be structured as intended.
4. Screen readers may interpret the document structure less clearly.

Structure less clearly.

5. These difference can affect layout, accessibility, and maintainability.

3. Identify missing semantic elements.

<header> - Defines the page heading

<main> - contains the main content

<section> - Groups related information

<footer> - Defines the footer content.

4. provide a corrected W3C - compliant HTML structure.

```
<title> company </title>
</head>
<body>
    <header>
      <h1> welcome </h1>
    </header>
    <main>
      <section>
          <h2> About us </h2>
          <p> welcome to our company website. </p>
      </section>
    </main>
  <footer>
     <p> copyright 2026 company </p>
  </footer>
</body>
</html>
```

5) Cross - Browser Compatibility

1. use <!DOCtype html>
2. close all Html tags properly
3. use semantic elements like <header>, <main>, and <section>
4. use the viewport meta tags for responsive design
5. test in Chrome, firefox, Edge, and Safari
6. use the W3C validator to check errors

3) Question: Form Submission Failure

1. Issues in form configuration

→ Button uses type = "button" instead of type = "submit"
→ missing required validation
→ Get method exposes password in the URL.

2. why HTTP request is not triggered?

→ The type = "button" does not submit the form automatically. use type="submit"

3. Role of Get method

→ Sends data through URL parameters
→ useful for searching information
→ not secure for passwords because data appears in the URL

4. corrected implementation

```
<!Doctype html>
<html lang = "en">
<head>
    <title> user login </title>
</head>
<body>
    <h1> user login </h1>
    <form action = "/submit" method = "post">
     <label> username : </label>
     <input type = "text" name = "username" required>

    <br> <br>
    <label> password : </label>
    <input type = "password" name = "password" required>
    <br> <br>
    <button type = "submit"> login </button>
    </form>
</body>
</html>
```

5. secure data transmission

1. use HTTPS to encrypt data
2. use post for password submission
3. Hash passwords before storing storing them
4. validate input on the server.
5. use secure sessions
6. never store plain-text passwords.